Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory work work #3.2

Signal Acquisition and Advanced Conditioning.

During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur Cravtov
std. gr. FAF-231

Verified by:
Alexei Martîniuc
asist. univ

Chișinău, 2026

Contents

1 Objective and Technical Analysis **1**
1.1 Signal Conditioning Techniques 2
1.1.1 Saturation (Clamping) 2
1.1.2 Median Filter (Salt-and-Pepper Noise) 2
1.1.3 Exponential Moving Average (EMA) 2

2 System Design **3**
2.1 System Architecture 3
2.2 Circuit Diagram 3
2.2.1 HC-SR04 Ultrasonic Distance Sensor Connections 4
2.2.2 NTC Thermistor Temperature Sensor Connections 5
2.2.3 Circuit Connection Summary 6
2.2.4 Practical Considerations 6
2.3 Conditioning Pipeline Flow 6

3 Implementation and Results **7**
3.1 Software Architecture 7
3.2 Layered Design 7
3.2.1 Distance Sensor Driver 7
3.2.2 Temperature Sensor Driver 8
3.3 Reporting and Visualization 9

4 Conclusion **10**

References **11**

A Source Code **11**

1 Objective and Technical Analysis

The objective of this laboratory work is to implement an advanced signal acquisition and conditioning pipeline for embedded systems. This involves not only reading raw data from sensors but also applying mathematical filters to remove noise and ensure data integrity. The system implements a three-stage conditioning pipeline: **Saturation**, **Median Filtering** (for salt-and-pepper noise), and **Exponential Moving Average (EMA)** for smoothing.

Through this work, the following learning outcomes are pursued:

- Implementing a multi-stage signal conditioning pipeline.

- Understanding and implementing non-linear filters (Median) and recursive linear filters (EMA).
- Managing intermediate data states (raw, saturated, median, filtered) in a multi-tasking environment.
- Real-time visualization of the conditioning process via serial interface and ASCII graphs.

1.1 Signal Conditioning Techniques

1.1.1 Saturation (Clamping)

Saturation is the simplest form of conditioning, ensuring that the input signal remains within a predefined physical or logical range $[min, max]$ [4]. It prevents outliers or sensor malfunctions from propagating "impossible" values through the system. This technique is also known as clamping or limiting in signal processing literature [5].

$$y[n] = \begin{cases} max & \text{if } x[n] > max \\ min & \text{if } x[n] < min \\ x[n] & \text{otherwise} \end{cases} \tag{1}$$

1.1.2 Median Filter (Salt-and-Pepper Noise)

The Median filter is a non-linear digital filtering technique, often used to remove noise while preserving edges [6]. It is particularly effective against "salt-and-pepper" noise (random isolated spikes) [7]. For a window of size N, the filter outputs the middle value of the sorted window samples. Unlike linear filters, the median filter does not blur sharp edges in the signal, making it ideal for applications where step changes are meaningful.

$$y[n] = \text{median}\{x[n], x[n-1], \ldots, x[n-N+1]\} \tag{2}$$

The computational complexity of median filtering is $O(N \log N)$ per sample when using sorting, though more efficient algorithms exist for small window sizes.

1.1.3 Exponential Moving Average (EMA)

EMA is a type of infinite impulse response (IIR) filter that applies weighting factors which decrease exponentially [8]. It is used to smooth the signal and reduce high-frequency noise [9]. The smoothing factor α (between 0 and 1) determines the balance between noise reduction and responsiveness: smaller values provide more smoothing but slower response to genuine changes.

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n-1] \tag{3}$$

The effective time constant of the EMA filter is $\tau = \frac{1-\alpha}{\alpha}$ samples. This recursive formulation requires only one multiplication, one subtraction, and one addition per sample, making it computationally efficient for embedded systems.

2 System Design

2.1 System Architecture

The system architecture follows a modular task-based design using FreeRTOS.

- **Acquisition Task**: Reads raw data from sensors (NTC Thermistor and Ultrasonic Sensor).
- **Conditioning Task**: Consumes raw values from a queue and applies the `SignalConditioner` pipeline.
- **Report Task**: Periodic reporting of the entire pipeline state with color-coded console output and ASCII graphs.

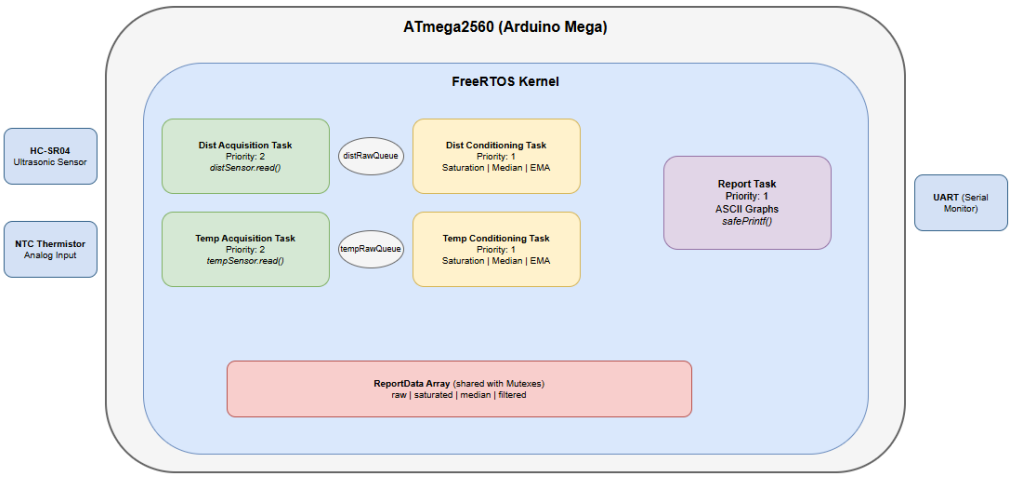

Figure 1: System Architecture Diagram for Lab 3.2

2.2 Circuit Diagram

Figure 2 shows the hardware connections between the ATmega2560 microcontroller and the sensors. The circuit consists of two main sensor interfaces connected to the microcontroller, each with distinct signal characteristics and interfacing requirements.

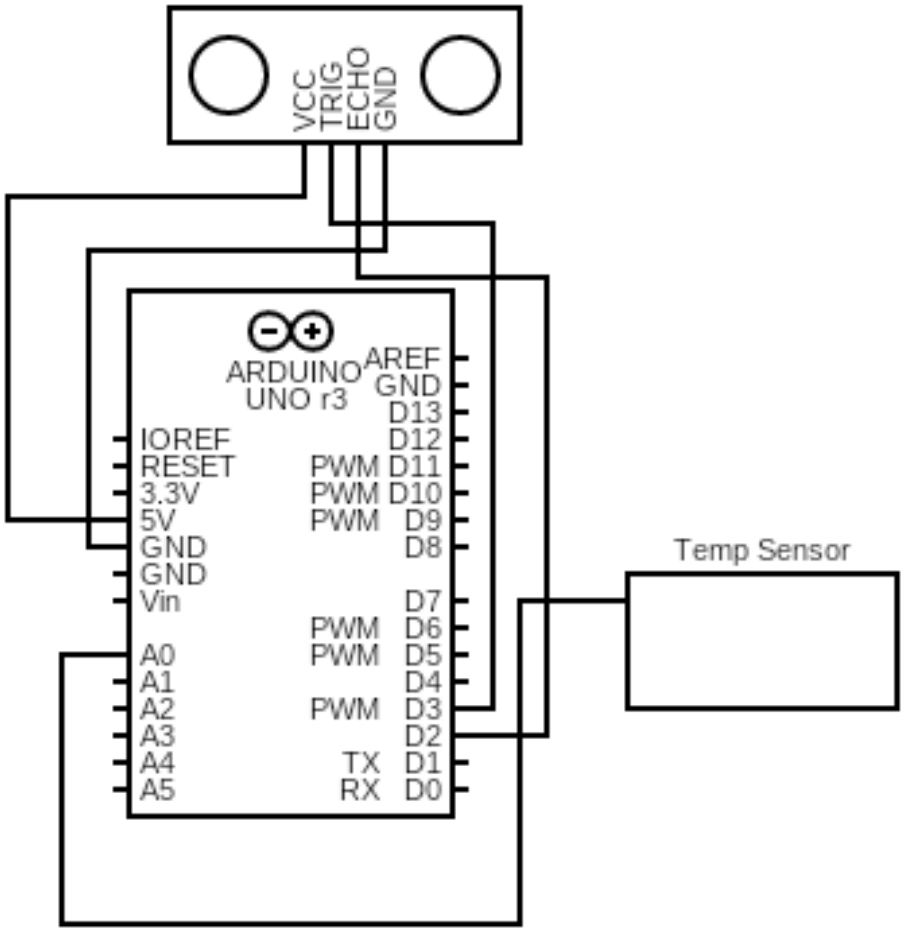

Figure 2: Circuit Diagram: Sensor Connections

2.2.1 HC-SR04 Ultrasonic Distance Sensor Connections

The HC-SR04 is a widely-used ultrasonic ranging module that provides 2cm to 400cm of non-contact measurement functionality with a ranging accuracy that can reach up to 3mm [2]. The sensor has four pins that connect directly to the microcontroller:

- **VCC**: Connected to the 5V power supply rail. The sensor requires a stable 5V supply for proper operation. The current consumption is approximately 15mA during measurement, making it suitable for battery-powered applications with appropriate power management.
- **GND**: Connected to the ground rail, providing the common reference potential for all signals. A common ground is essential for accurate timing measurements.
- **TRIG (Trigger)**: Connected to digital pin D3 of the ATmega2560. This input pin receives a 10µs HIGH pulse from the microcontroller to initiate a measurement cycle.

The sensor's internal circuitry detects the rising edge and begins the ultrasonic burst.

- **ECHO**: Connected to digital pin D2 of the ATmega2560. This output pin provides a pulse whose duration is proportional to the measured distance. The relationship is approximately 58µs per centimeter, derived from the speed of sound (343 m/s at 20°C).

The ultrasonic sensor operates on the time-of-flight principle. When triggered, it emits eight 40kHz ultrasonic pulses and simultaneously sets the ECHO pin HIGH. When the reflected sound waves are detected, the ECHO pin goes LOW. The distance is calculated as:

$$d = \frac{t_{echo} \times v_{sound}}{2} \tag{4}$$

where t_{echo} is the pulse duration and $v_{sound} \approx 343$ m/s. The division by 2 accounts for the round-trip of the sound wave.

2.2.2 NTC Thermistor Temperature Sensor Connections

The NTC (Negative Temperature Coefficient) thermistor is a semiconductor device whose resistance decreases predictably with increasing temperature [3]. It is connected in a voltage divider configuration:

- **Voltage Divider Circuit**: The NTC thermistor (nominal resistance 10kΩ at 25°C) is connected in series with a fixed 10kΩ resistor between VCC (5V) and GND. The choice of the fixed resistor value (equal to the thermistor's nominal resistance) provides maximum sensitivity near 25°C.

- **Output Signal**: The junction between the thermistor and the fixed resistor is connected to analog input pin A0 of the ATmega2560. The output voltage varies between approximately 0.5V (hot) and 4.5V (cold) within the operating range.

- **ADC Reference**: The internal 10-bit ADC uses AVCC (5V) as the reference voltage, providing a resolution of approximately 4.9mV per step. For temperature measurements, this translates to roughly 0.25°C resolution near room temperature.

The voltage divider arrangement allows the microcontroller to measure the thermistor's resistance indirectly through voltage measurement. The relationship between the output voltage V_{out} and the thermistor resistance R_{NTC} is given by:

$$V_{out} = V_{CC} \cdot \frac{R_{fixed}}{R_{NTC} + R_{fixed}} \tag{5}$$

As temperature increases, the NTC resistance decreases (hence "negative temperature coefficient"), causing the output voltage at A0 to rise. This behavior is characteristic of NTC thermistors and is exploited for temperature measurement.

2.2.3 Circuit Connection Summary

Table 1 summarizes the complete pin assignments for the sensor circuit. All connections are direct, without intermediate components such as level shifters, as all components operate at 5V logic levels compatible with the ATmega2560.

Table 1: Microcontroller Pin Assignments

Component	**Pin**	**Function**
HC-SR04 VCC	5V rail	Power supply
HC-SR04 GND	GND rail	Ground reference
HC-SR04 TRIG	D3	Trigger input (digital output)
HC-SR04 ECHO	D2	Echo output (digital input)
NTC Thermistor (top)	5V rail	Connected to VCC
NTC Thermistor (bottom)	A0 via junction	ADC input
10kΩ Fixed Resistor (top)	A0 via junction	ADC input
10kΩ Fixed Resistor (bottom)	GND rail	Ground reference

2.2.4 Practical Considerations

Several practical factors were considered in the circuit design:

- **Signal Noise**: Raw sensor readings contain inherent noise from various sources: acoustic reflections for the ultrasonic sensor and electrical interference for the thermistor circuit. The signal conditioning pipeline addresses these issues.
- **ADC Noise**: The thermistor circuit is susceptible to electrical noise, especially in environments with switching loads. A small capacitor (0.1µF) can be added between A0 and GND to filter high-frequency noise.
- **Self-heating**: Passing current through the thermistor causes self-heating, which can introduce measurement errors. The voltage divider current is approximately 250µA at 25°C, which minimizes self-heating effects.

2.3 Conditioning Pipeline Flow

Figure 3 illustrates the data flow through the `SignalConditioner` class. Each sample is first clamped, then stored in a circular buffer for median calculation, and finally passed through the EMA stage.

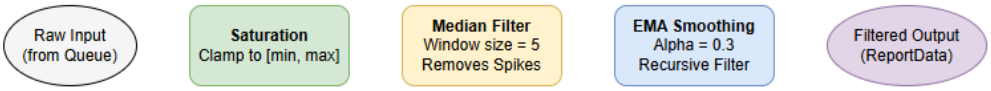

Figure 3: Signal Conditioning Pipeline Flowchart

3 Implementation and Results

3.1 Software Architecture

The `SignalConditioner` class encapsulates the mathematical logic, while the `ConditioningTask` manages the FreeRTOS integration.

- **Wait for Data**: Task blocks on `xQueueReceive` until a raw sample arrives.
- **Process**: Calls `conditioner.addSample(raw)`.
- **Shared State**: Updates the `ReportData` structure protected by a Mutex, storing all intermediate values (`raw`, `saturated`, `median`, `filtered`).

3.2 Layered Design

3.2.1 Distance Sensor Driver

Figure 4 shows the layer decomposition for the pulse-based digital sensor.

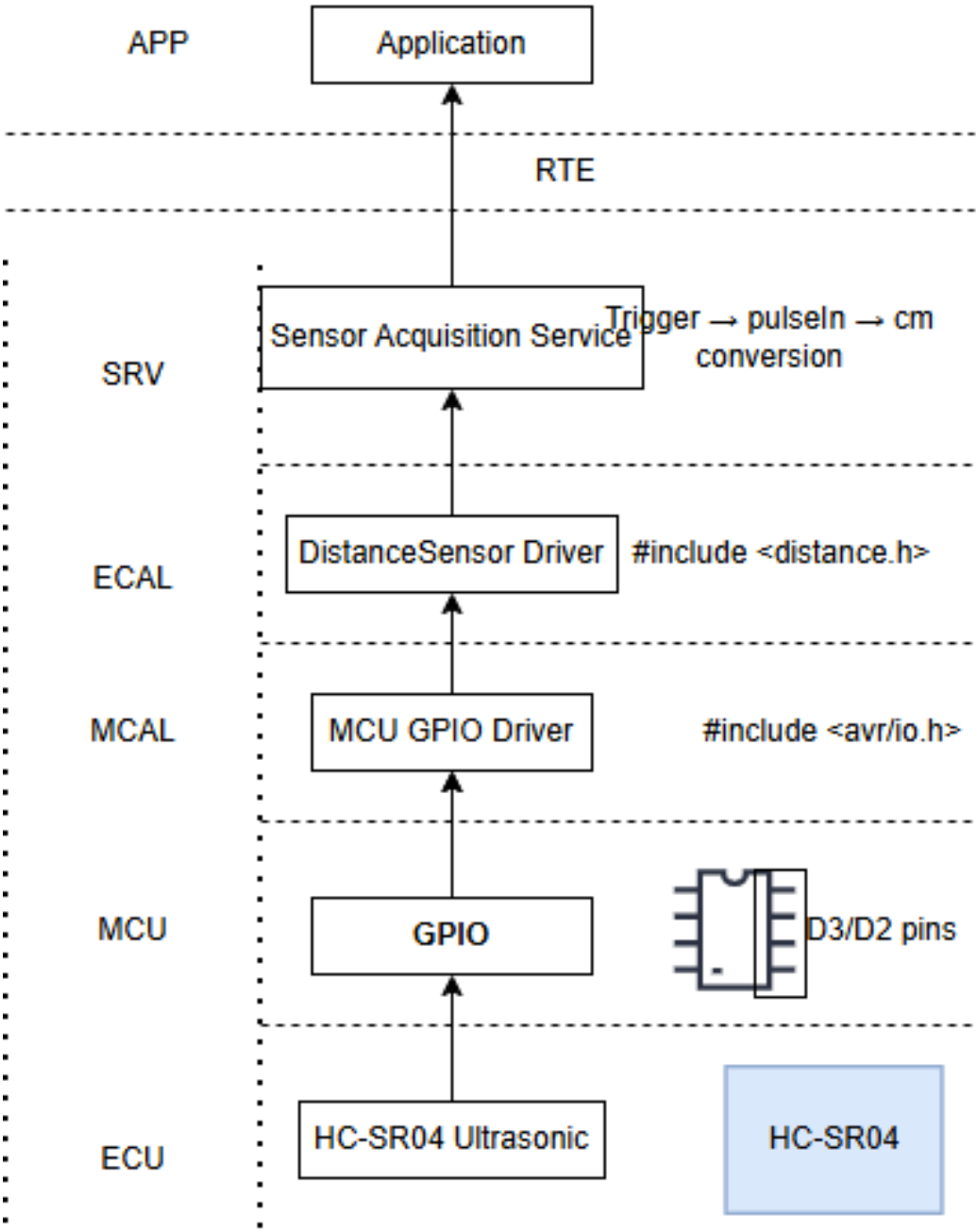

Figure 4: Distance Sensor layering

3.2.2 Temperature Sensor Driver

Figure 5 shows the layer decomposition for the analog sensor. The ECAL layer here includes the mathematical transformation from ADC counts to Celsius.

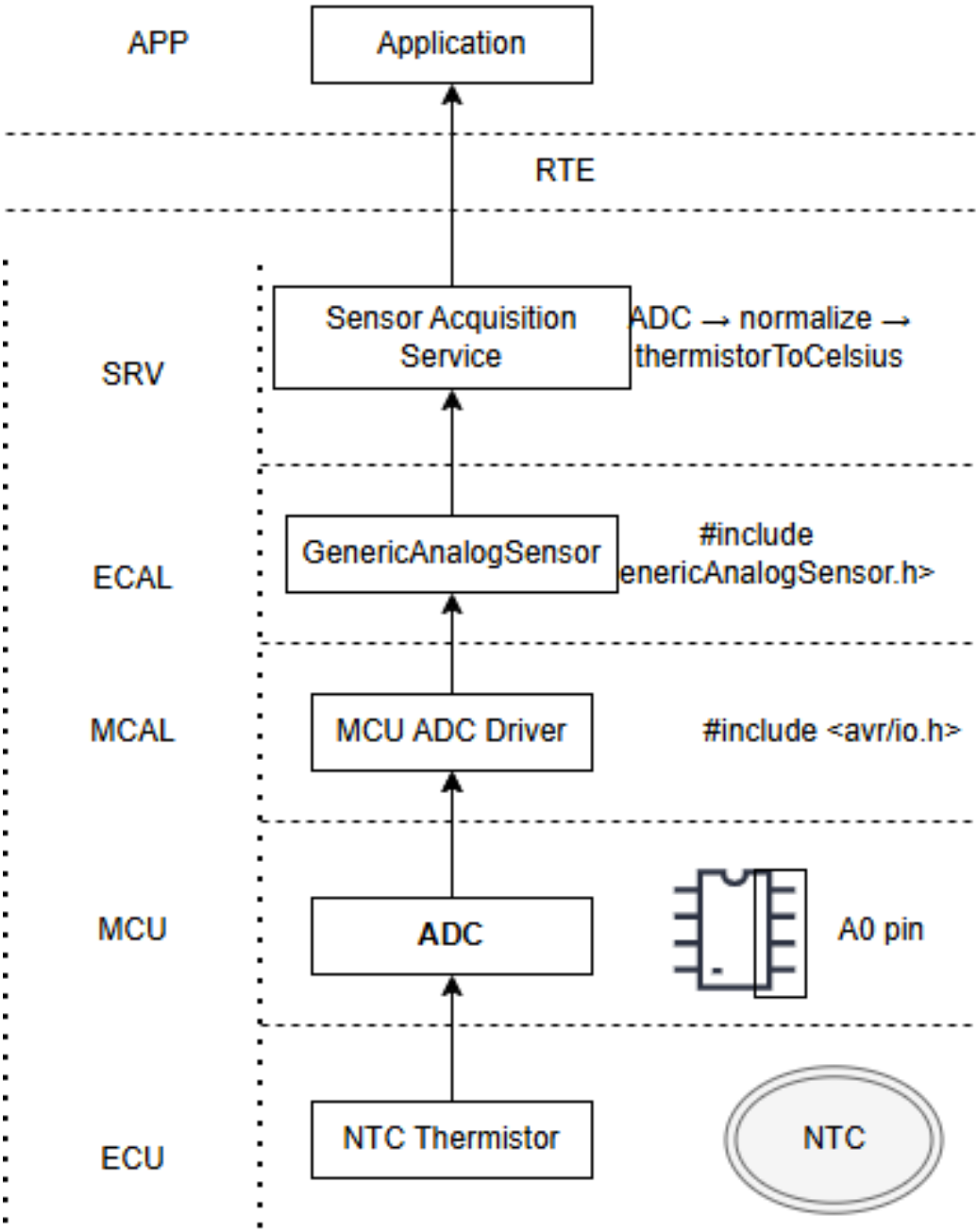

Figure 5: Temperature Sensor layering

3.3 Reporting and Visualization

A key feature of this implementation is the enhanced reporting. The `ReportTask` outputs a detailed breakdown of the pipeline for each sensor:

- **Status**: OK/LOW/HIGH based on saturation limits.
- **Pipeline Path**: Visual string showing $Raw \rightarrow Sat \rightarrow Med \rightarrow Filtered$.
- **ASCII Graph**: A dynamic bar representing the filtered value within its configured range.

The simulation results in Figure 6 demonstrate the effectiveness of the three-stage pipeline. Even when the raw sensor data contains significant noise or sudden spikes, the median filter effectively removes the outliers, and the EMA filter provides a smooth output suitable for control or further analysis.

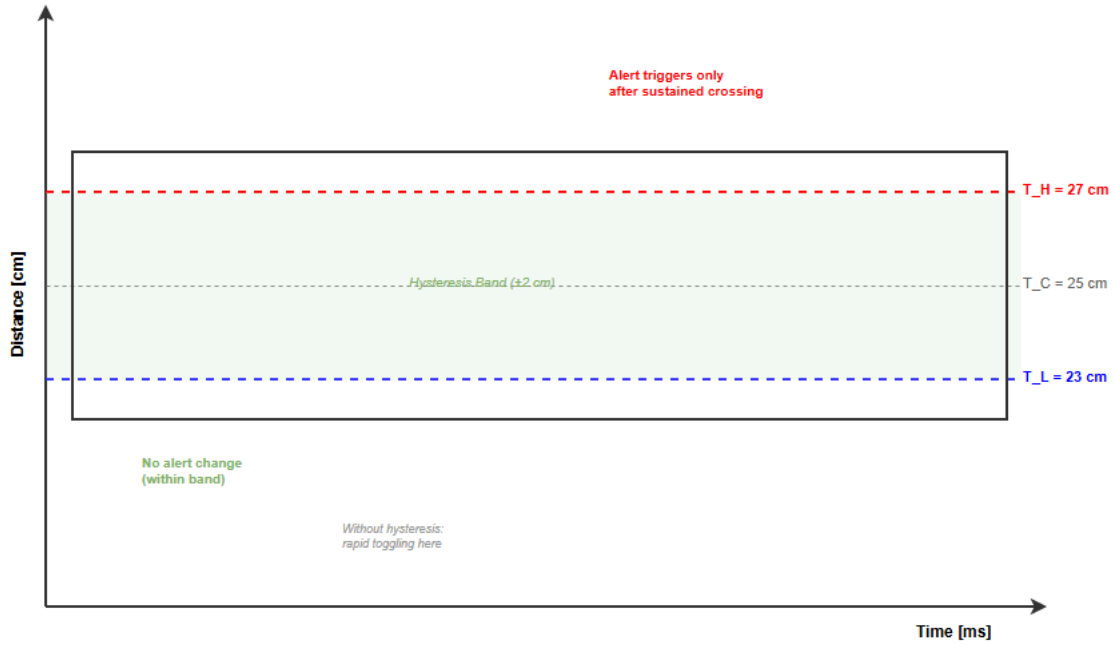

Figure 6: Console Visualization: Real-time Signal Conditioning Pipeline and ASCII Graphs

The output shows how the raw value is transformed through the pipeline:

1. **Raw**: The direct reading from the sensor.
2. **Sat (Saturated)**: Clamped to the $[min, max]$ range.
3. **Med (Median)**: The result of the median filter window, effectively removing "spikes".
4. **Flt (Filtered)**: The final output after the Exponential Moving Average smoothing.

The ASCII bar provides an intuitive visual representation of the current value relative to the sensor's full scale, allowing for quick system monitoring without complex external tools.

4 Conclusion

This laboratory work successfully implemented a robust signal conditioning system. By combining saturation, median filtering, and EMA, we created a pipeline that is resilient to both isolated noise spikes and continuous high-frequency fluctuations. The

modular architecture using FreeRTOS ensures that conditioning can run independently of acquisition, while the detailed reporting provides full visibility into the system's internal state.

References

[1] FreeRTOS API Reference, `https://www.freertos.org/a00106.html`

[2] HC-SR04 Ultrasonic Ranging Module Datasheet, `https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf`

[3] NTC Thermistor Principles and Applications, Vishay, `https://www.vishay.com/docs/29053/ntccomp.pdf`

[4] Lyons, R.G., "Understanding Digital Signal Processing", 3rd ed., Prentice Hall, 2010.

[5] Oppenheim, A.V., Schafer, R.W., "Discrete-Time Signal Processing", 3rd ed., Pearson, 2009.

[6] Huang, T.S. et al., "The Median Filter and Its Variations", in "Two-Dimensional Digital Signal Processing II", Springer, 1981.

[7] Gonzalez, R.C., Woods, R.E., "Digital Image Processing", 4th ed., Pearson, 2018.

[8] Proakis, J.G., Manolakis, D.G., "Digital Signal Processing: Principles, Algorithms, and Applications", 4th ed., Pearson, 2006.

[9] Smith, S.W., "The Scientist and Engineer's Guide to Digital Signal Processing", California Technical Publishing, 1997.

[10] Fraden, J., "Handbook of Modern Sensors: Physics, Designs, and Applications", 5th ed., Springer, 2016.

[11] Barr, M., Massa, A., "Programming Embedded Systems in C and C++", O'Reilly Media, 2006.

A Source Code

Full source code for Lab 3.2 is available in the repository.

`SignalConditioner.cpp`

```
#include "SignalConditioner.h"

```

```cpp
SignalConditioner::SignalConditioner(float minSat, float maxSat, uint8_t windowSize, float emaAlpha)
    : minSat(minSat), maxSat(maxSat), bufSize(windowSize),
      index(0), count(0), emaAlpha(emaAlpha), emaValue(0.0f), emaInitialised(false),
      lastRaw(0.0f), lastSaturated(0.0f), lastMedian(0.0f)
{
    buffer = new float[bufSize];
    for (uint8_t i = 0; i < bufSize; i++) buffer[i] = 0.0f;
}

float SignalConditioner::saturate(float value) const {
    if (value < minSat) return minSat;
    if (value > maxSat) return maxSat;
    return value;
}

float SignalConditioner::computeMedian() const {
    // Copy to temp array, then sort (bubble sort tiny N)
    uint8_t n = count < bufSize ? count : bufSize;
    float tmp[7]; // max window we ever use
    for (uint8_t i = 0; i < n; i++) tmp[i] = buffer[i];

    for (uint8_t i = 0; i < n - 1; i++) {
        for (uint8_t j = 0; j < n - i - 1; j++) {
            if (tmp[j] > tmp[j + 1]) {
                float t   = tmp[j];
                tmp[j]    = tmp[j + 1];
                tmp[j + 1] = t;
            }
        }
    }
    return tmp[n / 2];
}

void SignalConditioner::addSample(float raw) {
    lastRaw = raw;
    float sat = saturate(raw);
```

```
    lastSaturated = sat;
    buffer[index] = sat;
    index = (index + 1) % bufSize;
    if (count < bufSize) count++;

    // Compute median then apply EMA
    float median = computeMedian();
    lastMedian = median;
    if (!emaInitialised) {
        emaValue = median;
        emaInitialised = true;
    } else {
        emaValue = emaAlpha * median + (1.0f - emaAlpha) *
            emaValue;
    }
}

float SignalConditioner::getFiltered() const {
    return emaValue;
}

bool SignalConditioner::isReady() const {
    return count >= bufSize;
}

float SignalConditioner::getLastRaw() const        { return
   lastRaw; }
float SignalConditioner::getLastSaturated() const  { return
   lastSaturated; }
float SignalConditioner::getLastMedian() const     { return
   lastMedian; }
float SignalConditioner::getMin() const            { return
   minSat; }
float SignalConditioner::getMax() const            { return
   maxSat; }
```

Listing 1: SignalConditioner.cpp

`ConditioningTask.cpp`

```
#include "ConditioningTask.h"
```

```
#include <semphr.h>

ConditioningTask::ConditioningTask(SignalConditioner* conditioner)
    : conditioner(conditioner), inQueue(NULL), reportEntry(NULL) {}

void ConditioningTask::setInQueue(QueueHandle_t q)     { inQueue = q; }
void ConditioningTask::setReportEntry(ReportData* r)   { reportEntry = r; }

void ConditioningTask::run() {
    while (true) {
        float raw = 0.0f;
        if (inQueue != NULL && xQueueReceive(inQueue, &raw, portMAX_DELAY) == pdTRUE) {
            conditioner->addSample(raw);

            // Update the shared report entry with the conditioned values
            if (reportEntry != NULL && reportEntry->xSemaphore != NULL) {
                xSemaphoreTake(reportEntry->xSemaphore, portMAX_DELAY);
                reportEntry->raw          = conditioner->getLastRaw();
                reportEntry->saturated    = conditioner->getLastSaturated();
                reportEntry->median       = conditioner->getLastMedian();
                reportEntry->filtered     = conditioner->getFiltered();

                // Simple status check check against min/max saturation
                if (reportEntry->raw < conditioner->getMin())
                    reportEntry->status = -1;
```

```
                    else if (reportEntry->raw > conditioner->getMax
                        ()) reportEntry->status = 1;
                    else reportEntry->status = 0;

                    xSemaphoreGive(reportEntry->xSemaphore);
                }
            }
        }
}
```

Listing 2: ConditioningTask.cpp

`ReportTask.cpp`

```
#include "ReportTask.h"
#include "SafePrintf.h"
#include <stdlib.h>
#include <Arduino.h>

ReportTask::ReportTask(ReportData* data, uint8_t numberOfSensors
    , uint32_t delay)
    : reportData(data), numberOfSensors(numberOfSensors), delay(
        delay) {}

void ReportTask::run() {
    static char fRaw[10], fSat[10], fMed[10], fFil[10];

    while (true) {
        // ── Human-readable & Colorful Pipeline Report ──
        for (uint8_t i = 0; i < numberOfSensors; i++) {
            if (reportData[i].xSemaphore != NULL) {
                xSemaphoreTake(reportData[i].xSemaphore,
                    portMAX_DELAY);

                int8_t s = reportData[i].status;
                float r = reportData[i].raw;
                float sa = reportData[i].saturated;
                float m = reportData[i].median;
                float f = reportData[i].filtered;
                float mn = reportData[i].min;
                float mx = reportData[i].max;
```

```

            xSemaphoreGive(reportData[i].xSemaphore);

            dtostrf(r, 6, 2, fRaw);
            dtostrf(sa, 6, 2, fSat);
            dtostrf(m, 6, 2, fMed);
            dtostrf(f, 6, 2, fFil);

            // ── Fancy Console Output ──
            // \x1b[32m - Green, \x1b[31m - Red, \x1b[33m - Yellow, \x1b[36m - Cyan, \x1b[0m - Reset

            const char* statusColor = "\x1b[32m"; // Green OK
            const char* statusStr = "OK";
            const char* barColor = "\x1b[32m";

            if (s < 0) {
                statusColor = "\x1b[33m"; // Yellow
                statusStr = "LOW";
                barColor = "\x1b[33m";
            } else if (s > 0) {
                statusColor = "\x1b[31m"; // Red
                statusStr = "HIGH";
                barColor = "\x1b[31m";
            }

            safePrintf("\x1b[1m[%s]\x1b[0m Status: %s%s\x1b[0m | Value: \x1b[36m%s\x1b[0m\n",
                reportData[i].sensorName, statusColor, statusStr, fFil);

            safePrintf("  Pipeline: Raw:%s -> Sat:%s -> Med:%s -> Filtered:%s\n",
                fRaw, fSat, fMed, fFil);

            // ── ASCII Graph ──
            int width = 20;
```

```
                float range = mx - mn;
                float percentage = (range > 0.001f) ? (f - mn) / range : 0.0f;
                if (percentage < 0.0f) percentage = 0.0f;
                if (percentage > 1.0f) percentage = 1.0f;
                int pos = (int)(percentage * width);

                safePrintf("  Graph: [");
                for (int j = 0; j < width; j++) {
                    if (j < pos) safePrintf("%s#\x1b[0m", barColor);
                    else if (j == pos) safePrintf("\x1b[33m|\x1b[0m");
                    else safePrintf("-");
                }
                safePrintf("] %d%%\n", (int)(percentage * 100));
            }
        }

        // --- Teleplot output (>label:value format) ---
        for (uint8_t i = 0; i < numberOfSensors; i++) {
            if (reportData[i].xSemaphore != NULL) {
                xSemaphoreTake(reportData[i].xSemaphore, portMAX_DELAY);
                float f = reportData[i].filtered;
                xSemaphoreGive(reportData[i].xSemaphore);

                dtostrf(f, 6, 2, fFil);
                safePrintf(">%s:%s\n", reportData[i].sensorName, fFil);
            }
        }

        safePrintf("------------------------------------------------------------\n");

        vTaskDelay(pdMS_TO_TICKS(delay));
```

```
        }
}
```

Listing 3: ReportTask.cpp